

* Version → ~~17~~ java 17 ^{→ stable version} onwards

* Types of comments

* Java Data types → 1) Primitives
2) User defined

* Conversion → Implicit & Explicit

* Note {
char + char =
byte + byte =
short + short = } int

* Binary Operations → 1) One's complement

2) Two's complement

3) Left Shift

3) Right Shift

4) Unsigned Right Shift

* ~~Java~~ Number Systems → 1) Octal (8) (0-7)

2) Hexadecimal (16) (0-9-A-F)

3) Decimal (10) (0-9)

4) Binary (2) (0-1)

- MSB (Most Significant Bit)

- LSB (Least Significant Bit)

* Bitwise Operator → 1) AND

2) OR

3) XOR

* Java Info →

* Java Features

* Internationalization

* Procedural Oriented Programming

* Object Oriented Programming

* Class definition

* Instance member

* Member function

* NullPointerException

②

Page No.:

Date: / /

- * Local Variable
- * Constructor → 1) Default / No args
2) Parameterized
- * Method → ~~Overloading~~ / Function
- * Finalize() method
- * Static Member
- * final keyword → (applied to...?)
- * Class Loader → 1) Bootstrap / Premordial
2) Extension
3) System } types of class loader
- * Static ~~block~~ block or initializer
- * Static block VS Static Member Function
- * Creating Object (Instantiating a class)
- * Counting Objects
- * Non Static block
- * SOP (System.out.println) Info →
- * Garbage Collection, Automatic Garbage Collection
- * Default value of each datatype
- * this keyword
- * Singleton class
- * Lazy & eager Resolution
- * Scanner class
- * What is class Class?
- * Named Object
- * Anonymous Object
- * Array → 1) 1D Array
2) 2D Array
3) Jagged Array

③

- * Array of object instance.
- * Package
- * Jar
- * JDK
- * JVM
- * How to do Documentation?
- * DLL
- * Relationship bet Developer & Client
- * Developer Client Application Steps
- * Reusability
- * Inheritance →
 - 1) Single level
 - 2) Multi-level
 - 3) Hierarchical
- * Super keyword
- * Access Modifiers →
 - 1) Private
 - 2) Public
 - 3) Protected
 - 4) Default
- * Method Overriding
- * Method Overloading
- * Polymorphism
- * Overriding Rules
- * Covariant
- * Dynamic Polymorphism
- * UpCasting
- * DownCasting
- * Binding →
 - 1) late binding
 - 2) early binding

- * Maintenance Problem & Drawback
- * instanceof operator
- * Abstract class
- * Abstract function
- * Abstraction → 1) no abstraction
2) partial abstraction
3) 100% abstraction
- * Document Comment
- * Interfaces
- * Reference
- * ClassCastException
- * Static block invocation
- * 4 pillars of OOPs → 1) Abstraction
2) Encapsulation
3) Polymorphism
4) Inheritance
- * Encapsulation
- * Encapsulation VS Abstraction
- * Constructor invocation by Is a / Has a
- * Primitive VS Reference type Conversion
- * Inheritance drawbacks
- * Composition Advantages
- * Program to Interface
- * Program to Implementation
- * Association
- * Aggregation
- * super VS this
- * Call stack

5

- * Object class
- * Object method
- * "Equals()" VS "==" operator
- * hashCode contract
- * Enum
- * Without enum, With enum
- * String
- * Immutable, Mutable String
- * StringBuilder
- * Nested classes
- * Inner() method
- * Wrapper classes
- * Autoboxing & Unboxing
- * Ellipsis
- * intern() method
- * equalsIgnoreCase() method
- * Outer. inner class
- * static nested class
- * Anonymous inner class
- * parseInt() & trim() method
- * Types of methods → 1) Fixed argument method
2) Variable no argument method
- * TypeCasting.
- * Exception Handling
- * try-catch-finally
- * throw keyword
- * throws keyword
- * throw VS throws
- * handle & declare rules

6

Page No.:

Date: / /

- * Command line arguments
- * Widening.
- * Bridge method in Java
- * Throwable Class
- * Error VS Exception
- * Runtime Exception
- * Assertions.
- * User defined exceptions
- * NoClassDefFoundError VS ClassNotFoundException
- * Finally block
- * Exceptions types → 1) Checked
2) Unchecked.
- * forName()
- * ARM block
- * printStackTrace()
- * Overriding ~~rule~~ & Checked exception Rule
- * Clone method.
- * Object cloning.
- * CloneNotSupportedException
- * Background for copy
- * Super.clone()
- * Marker Interface.
- * Shallow Copy.
- * Ways to create copy of objects
- * How to create Immutable class in Java
- * Reflection API
- * newInstance(object) → 1) With Reflection
2) Without Reflection } vs.

⑦

- * Multithreading → 1) process-based multi-tasking
2) thread-based multi-tasking.
- * Process-based VS thread-based
- * Thread Scheduler → 1) Preemptive
2) Time Slicing
- * Thread class & methods
- * Runnable Interface
- * Ways to create Multithreading appl → 1) extends Thread
2) implements Runnable
- * Synchronization
- * Synchronized keyword.
- * Object lock
- * Class lock
- * Advance Multithreading.
- * Thread Pool
- * ReentrantLock class
- * thread safe class
- * Executors
- * wait VS sleep
- * Thread lifecycle
- * Daemon threads.
- * Difference bet ReentrantLock VS Synchronized
- * ~~By~~ Serialization
- * Deserialization
- * Serializable Interface
- * Object Graph
- * serialVersionUID
- * readObject() & writeObject()

- * Externalization
- * ~~ObjectOutputStream~~ Customized Serialization
- * File Handling
- * Object Persistence
- * Stream → 1) InputStream
 - 2) OutputStream
- * ByteStreams
- * Unicode Character Stream
- * absolute Path
- * Random Access File
- * Object Persistence
- * transient keyword
- * Generics
- * Collection API → ~~Array~~
 - 2) List
 - 3) Queue
 - 4) Stack
 - 5) Map
 - 6) Set
- * List → 1) ArrayList
 - 2) LinkedList
 - 3) Vector
 - 4) Copy on write ArrayList
- * Queue
- * Stack
- * Iterator
- * ConcurrentModificationException
- * Algorithms

9

* Map → 1) TreeMap
2) HashMap
3) Hashtable

* Set → 1) HashSet
2) TreeSet
3) LinkedHashSet

* load & capacity

* hasNext() & next()

* hashCode()

* put operation

* get operation

* Hashtable

* Properties Class

* Collection VS Collections

* Comparable (compareTo)

* Comparator

* Iterator with set

* Synchronized collection.

* IndexOutOfBoundsException

* Thread Safety

* Synchronize block

* ArrayStoreException

* Socket Programming

* Port No.

* Layers of Network

* TCP / UDP

* Application layers

* JRMIP, IIOP

* Client VS Server

- * Java-8 Info
- * Lambda
- * Functional Interface.
- * Method Reference.
- * Abstract method arguments.
- * Stream API
- * Predicate
- * peek() method
- * Changes in Map → 1) putIfAbsent
2) replace VS put
3) computeIfAbsent
- * reduce() method
- * NIO (New Input Output)
- * Buffer class
- * Locale class
- * Optional <T>
- * Pipelined